

TP2 – Geometrical Image Recreation

1 End Goal

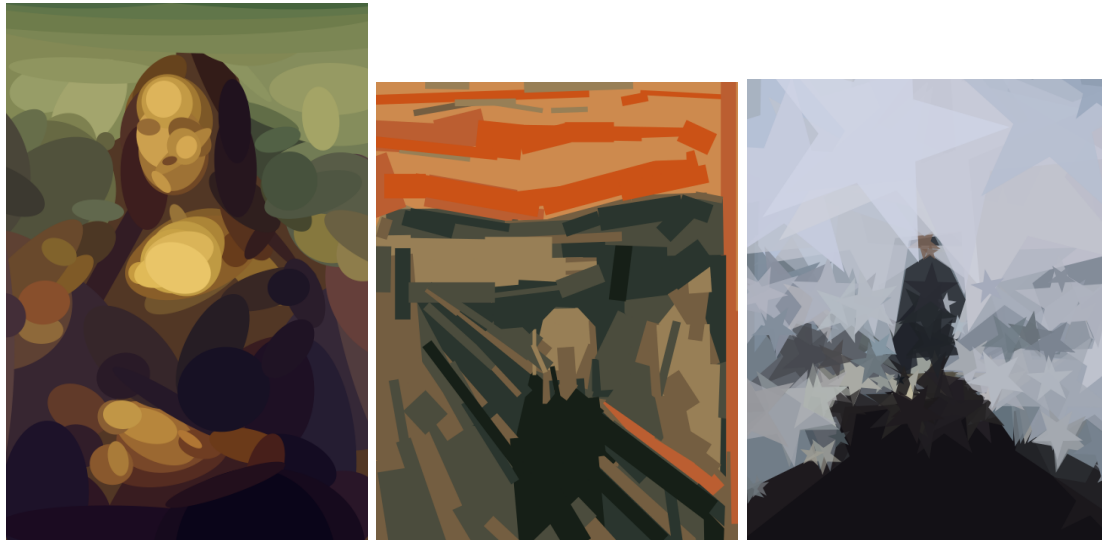


Figure 1: **left:** *Mona Lisa*, 118 ellipses; **middle:** *The Scream*, 141 rotated rectangles and 8 colours; **right:** *Wanderer above the Sea of Fog*, 104 stars and alpha transparency.

We aim to approximate images using a limited set of geometric shapes.¹ Specifically, we want to develop a pipeline that, given a source image and a set of constraints (such as the types of allowed shapes, a minimum or maximum number of shapes, or the number of colours), can efficiently generate an SVG image that closely resembles the original.

More precisely, we want a *command line interface* in Python that takes in input an image and outputs an “aesthetically pleasing” approximation. Whilst we expect some constraint to be specifiable, the core application design and implementation details are left entirely to your discretion. **Only rule:** no copy/paste from GitHub.

Use case example:

```
python3 png2svg.py --shape square --n 100 --time 600 \  
  --input monalisa.png --output masterpiece.svg
```

2 Objectives

Objectives of this practical work include:

- formulating and solving a complex real-world optimisation problem;
- implementing an effective and efficient metaheuristic approach;
- exploring the impact of parameter-tuning on metaheuristics.

¹More examples on: <http://www0.cs.ucl.ac.uk/staff/a.blot/art/>

3 Preliminaries

Problem Modelling

Take a moment to think about and clearly define the problem.

- what data do you have access to?
- what are your decision variables?
- what is the objective function?
- what are the constraints?

SVG Generation

Your application will need to output standalone SVG files. As such, you will need to understand basic SVG structure, including at minimum the syntax for the main `<svg>` node, the shape `<rect>`, `<circle>`, and `<ellipse>` nodes, as well as basic understanding of colouring and transformations.

Useful crash course:

🔗 <https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial>

Image Manipulation

There are many Python image processing libraries. You can use PIL (Pillow, 🔗 <https://python-pillow.org/>), which is included on the ISTIC machines.² Ensure you have a basic understanding of its pixel manipulation capabilities. For example, what does the following snippet achieve?

```
from PIL import Image

input_image = Image.open('monalisa.png')
output_image = Image.new('RGB', input_image.size)

width = input_image.width
height = input_image.height

for x in range(width):
    for y in range(height):
        pixel = input_image.getpixel((width-x-1, height-y-1))
        output_image.putpixel((x, y), pixel)

output_image.save('output.png')
```

Command Line Interface

You will use the `argparse` library, also available on the ISTIC machines (🔗 <https://docs.python.org/3/library/argparse.html>).

As an exercise, take the above PIL example and add two arguments “`-input`” and “`-output`” to avoid hard-coding the two raw strings “`monalisa.png`” and “`output.png`”.

²All throughout this TP, go through me before using any non-native Python library

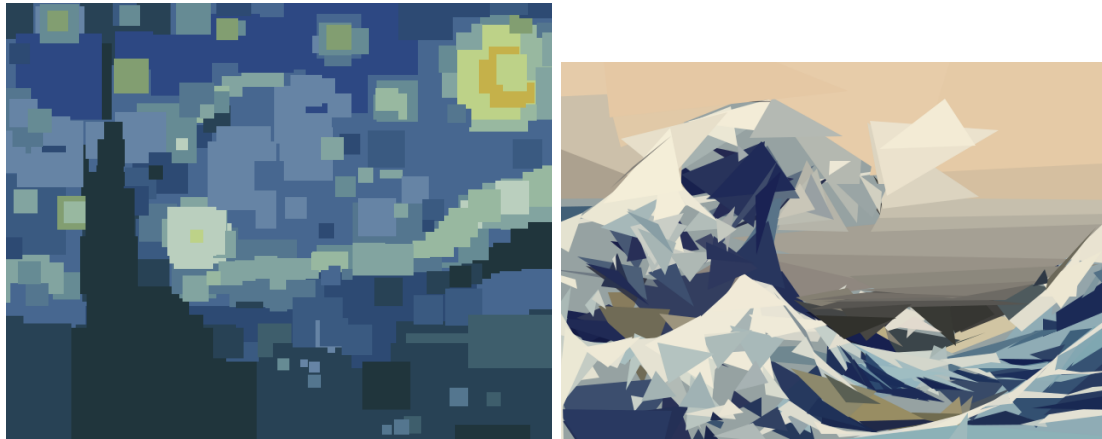


Figure 2: **left:** *Starry Night*, 215 squares and 16 colours; **right:** *The Great Wave off Kanagawa*, 241 triangles.

Solution Representation

You will need to encode this problem, yourself, from scratch, in Python. To approach it effectively, it will be useful to clearly distinguish between two representations of a solution: its *genotype* and its *phenotype*. The *genotype* is the simplified underlying representation of the object you want to represent (think of it of its “DNA”). Here, it might be a collection of geometric shapes, each with location and colour data—perhaps even reduced to a simple numeric array. On the other hand, the *phenotype* is the visible form of the solution, required to compute its fitness. In our case, this will likely be a matrix of pixels to compare with the original image.

Before continuing, make sure to take the time to think about which data structure you want to choose to manipulate both genotype and phenotype.

Useful auxiliary functions to implement might include:

- computing an SVG from a genotype;
- computing a PNG from a phenotype;
- computing a phenotype from a genotype;
- computing a fitness value from a phenotype;
- mutating a genotype;
- performing a crossover between two genotypes.

4 *Suggested* Plan of Action

Code a skeleton application

First, focus on the overall code structure and solution representation. Write a first version of your main script that loads the target image, create a “blank” approximation and compute its fitness without implementing any improvement algorithm yet. Test your solution representation by generating a random solution with one simple geometric shape (e.g., one square aligned with the pixel grid) to ensure the code structure works as expected.

Implement your first metaheuristic

Then, rather than stopping after the first empty or random solution, implement a random search by repeatedly generating new random solutions and computing their fitness, creating a basic working main loop. From there, add a simple local search mutation operator, devise a stopping criterion, and introduce a perturbation operator. This will give you a simple iterated local search.

Implement additional functionalities

Next, expand your code by adding more advanced features. Implement crossover operators to combine different solutions and implement a population-based metaheuristic. Add support for multiple geometric shapes, multiple stopping conditions, etc. Consider implementing or fine-tuning problem-specific strategies, such as for example initial solution generation or mutation. Add any functionality you see fit, as for example the possibility to restart from a previously found solution.

Parameter tuning and efficiency analysis

Finally, focus on refining your algorithm's performance. Study on a few examples the anytime behaviour of your application (i.e., trade-off between computation time and expected efficiency). For the main design decisions introduced (at the very least, ILS vs GA), report on the performance differences and adjust the default parameters.

5 Technical Specifications

The following criteria are meant to to guide—not constrain—you. Nothing is set in stone; bring your own ideas and check with me for individual guidance.

Functionalities

minimal: single simple shape type (e.g., square/rectangle)

expected: multiple shape types (e.g., rotations, circle/triangle/polygon)

advanced: restart from saved solution

Algorithm design

minimal: simple local search (e.g., first improvement), fixed solution size

expected: simple genetic algorithm

advanced: ad-hoc strategies, initial constructive heuristic, variable solution size

Software engineering

minimal: git, tests, function-level documentation

expected: modular code design, clear structure

advanced: nice user interface (CLI, console outputs)

Final report (README.md/pdf)

minimal: user manual, output examples (images)

expected: efficiency analysis, reflection on design choices

advanced: comparisons of algorithms/variants, ablation analysis

Minimal: 10/20, expected: 14/20, advanced: 18/20, anything beyond: bonus.